INCIDENT INVESTIGATION & RESOLUTION REPORT

INC-1043 · Payment summary endpoint failing after v3.8.1 release · Mission MSN-016BCDE1

Outcome: RESOLVED (confirmed by a human)

Service	Environment	Severity	Iterations	Agents used	Tool calls	Requested by
payment-api	Sandbox	High	1 of 3	9	30	Alex Morgan

The Orchestrator ran 1 iteration with a team chosen for this ticket. 9 agent findings were produced using 30 read-only tool calls (reading evidence, logs, status, changes, source files and runbooks).

Where the findings came from: 9 × Foundry agent.

Final assessment: MEASURED: 0/50 requests failed with HTTP 5xx (0.0% error rate) at 50 concurrent requests; p95 943 ms, max 946 ms; release 3.8.1, pool size 20; Feature X (GET /payments/summary) available.

1. The issue (the input we were given)

Description: Under concurrent load, GET /payments/summary returns HTTP 500 for a share of requests in the isolated Payment API sandbox. Run the calibrated workload to measure it live.

Business impact: Feature X (payment summary) fails for a share of requests under load; the previous release did not have this endpoint.

Observed error: sqlalchemy.exc.TimeoutError: QueuePool limit of size 5 overflow 0 reached, connection timed out, timeout 2.00

Expected behaviour: 50 concurrent summary requests with 0 HTTP 5xx and p95 under 2 s

ID	Type	Evidence	Classification
`EVD-421`	Change	Payment API v3.8.1 release configuration	Observed
`EVD-422`	Document	Sandbox service-level target	Observed
`EVD-E969AB54`	Metric	Measured workload: 9/50 requests failed (pool 5)	Observed
`EVD-20E68B2D`	Metric	Measured workload: 0/50 requests failed (pool 20)	Observed
`EVD-B28F2E14`	Metric	Measured workload: 0/50 requests failed (pool 20)	Observed
`EVD-8D835B12`	Metric	Measured workload: 0/50 requests failed (pool 20)	Observed

2. What the agents did, iteration by iteration

Iteration 1

Orchestrator's reasoning (Foundry agent): The incident involves HTTP 500 errors under concurrent load due to DB connection pool exhaustion after reducing pool size in v3.8.1 (evidenced by EVD-421 and error message). Metrics confirm failures at pool size 5 but none at size 20 (EVD-E969AB54 vs EVD-20E68B2D, EVD-B28F2E14). The team must investigate configuration and logs to isolate root cause and then propose and validate a remediation, followed by risk assessment and release planning.

Agent	Why it was chosen	Source	Confidence
Intake & classification	Validate incident severity, scope, and SLA given high-impact concurrent load failures in Sandbox.	Foundry agent	0.95
Log analysis	Analyze logs to confirm error patterns and correlate 500 errors with DB pool exhaustion from evidence.	Foundry agent	0.95
Observability correlation	Examine latency distributions and error rates to verify impact of pool size on performance and errors.	Foundry agent	0.95
Repository analysis	Verify timeline and changes introduced in release v3.8.1, confirming correlation with error onset.	Foundry agent	0.95
Root-cause analysis	Synthesize evidence (logs, metrics, repo) to identify root cause as DB pool too small causing timeouts.	Foundry agent	0.95

Agent	Why it was chosen	Source	Confidence
Remediation	Propose configuration fix (increase DB pool size) to resolve connection timeouts and meet SLA targets.	Foundry agent	0.98
Test engineering	Define concurrency workload tests to validate fix under 50 concurrent requests with no 5xx errors.	Foundry agent	0.95
Change risk	Evaluate risk of increasing DB pool size and ensuring no negative impact on Sandbox environment stability.	Foundry agent	0.95
Release coordination	Plan controlled deployment of the configuration fix with separation of duties and audit record keeping.	Foundry agent	0.95

Left out on purpose: knowledge (No past runbooks or post-mortems indicated in evidence; root cause likely configuration-based.); security (No indications of security issues or data leak risks from the incident or proposed fix.); validation (Validation needed only after remediation proposal and test results; premature at this stage.); summary (Incident summary drafting follows resolution and validation stages to capture complete findings.)

3. Investigation trail: what each agent looked at and concluded

Intake & classification · iteration 1 · AI-GENERATED / Foundry agent behave-intake v4

Tool	Input	Result
`get_ticket`	-	ticket read
`list_evidence`	-	5 evidence

Finding: The Payment API v3.8.1 release introduced Feature X (GET /payments/summary) with a DB connection pool size reduced to 5, causing HTTP 500 errors under 50 concurrent requests due to pool timeouts (9/50 requests failed). Increasing the pool size to 20 eliminates the failures and meets the sandbox SLO of zero HTTP 5xx and p95 under 2 seconds. Severity High classification is appropriate given the impact and failure rate. Recommended recovery includes increasing pool size to 20 or rolling back the release. *(confidence 0.95; evidence cited: EVD-421, EVD-422, EVD-E969AB54, EVD-20E68B2D, EVD-B28F2E14)*

Log analysis · iteration 1 · AI-GENERATED / Foundry agent behave-logs v4

Tool	Input	Result
`list_evidence`	-	5 evidence
`search_logs`	TimeoutError	0 matches
`search_logs`	QueuePool	0 matches

Finding: The Payment API v3.8.1 release introduced Feature X and reduced the DB connection pool size to 5 with no overflow and a 2s timeout. Under 50 concurrent requests, this configuration caused an 18% HTTP 500 failure rate correlated with connection pool timeouts, while increasing the pool size to 20 resolved the errors. The root cause is the insufficient DB connection pool size leading to queue pool timeout errors during concurrent load. *(confidence 0.95; evidence cited: EVD-421, EVD-E969AB54, EVD-20E68B2D, EVD-B28F2E14)*

Observability correlation · iteration 1 · AI-GENERATED / Foundry agent behave-observability v4

Tool	Input	Result
`get_service_status`	-	status read
`search_logs`	QueuePool	0 matches
`search_logs`	TimeoutError	0 matches

Finding: The high error rate (18%) and elevated p95 latency (~2444 ms) for GET /payments/summary under 50 concurrent requests are caused by connection pool saturation. The Payment API v3.8.1 was configured with PAYMENT_DB_POOL_SIZE=5 and max_overflow=0, leading to frequent connection timeouts (QueuePool limit reached). Increasing the pool size to 20 eliminates the 5xx errors and reduces latency to ~960 ms, meeting service targets. The root cause is the insufficient DB connection pool size for the new feature workload on v3.8.1. *(confidence 0.95; evidence cited: EVD-421, EVD-E969AB54, EVD-20E68B2D, EVD-B28F2E14)*

Repository analysis · iteration 1 · AI-GENERATED / Foundry agent behave-repository v4

Tool	Input	Result
`list_changes`	-	5 changes
`list_repo_files`	-	4 files
`read_repo_file`	sandbox.env.example	sandbox.env.example read
`read_repo_file`	app.py	app.py read

Finding: The Payment API v3.8.1 release introduced Feature X which requires holding a DB connection for approximately 0.3 seconds per request. The database connection pool size was reduced to 5 with max_overflow set to 0 and pool_timeout to 2 seconds. Under a 50 concurrent request workload, this configuration leads to an 18% HTTP 500 error rate caused by SQLAlchemy QueuePool timeouts, whereas increasing the pool size to 20 eliminates these failures. This suggests that the root cause of the endpoint failures under load is the insufficient connection pool size configuration in v3.8.1, correlating directly with the onset of errors after deployment. *(confidence 0.95; evidence cited: EVD-421, EVD-E969AB54, EVD-20E68B2D, EVD-B28F2E14)*

Root-cause analysis · iteration 1 · AI-GENERATED / Foundry agent behave-root-cause v4

Tool	Input	Result
`list_evidence`	-	5 evidence
`search_logs`	TimeoutError	0 matches
`list_changes`	-	5 changes
`get_service_status`	-	status read
`list_repo_files`	-	4 files
`search_knowledge`	connection pool timeout	2 articles

Finding: The root cause of the Payment API v3.8.1 /payments/summary endpoint failures under concurrent load is the reduction of the database connection pool size from 20 to 5, with no overflow and a 2-second timeout, causing frequent QueuePool timeouts and HTTP 500 errors (18% failure rate) under 50 concurrent requests. Increasing the pool size back to 20 fully eliminates the errors and meets the sandbox service-level target. This is supported by evidence from configuration (EVD-421), workload metrics showing failures at pool size 5 and no failures at pool size 20 (EVD-E969AB54, EVD-20E68B2D, EVD-B28F2E14), and documented recovery options (EVD-422, KB-108). *(confidence 0.95; evidence cited: EVD-421, EVD-422, EVD-E969AB54, EVD-20E68B2D, EVD-B28F2E14)*

Remediation · iteration 1 · AI-GENERATED / Foundry agent behave-remediation v4

Tool	Input	Result
`list_changes`	-	5 changes
`get_service_status`	-	status read
`read_repo_file`	config.py	File 'config.py' is not available. Use list_repo_files.
`list_repo_files`	-	4 files
`read_repo_file`	sandbox.env.example	sandbox.env.example read
`read_repo_file`	app.py	app.py read

Finding: The root cause of the Payment API v3.8.1 /payments/summary endpoint failures (HTTP 500) under concurrent load is due to the database connection pool size being set to 5 with max_overflow 0, leading to connection pool exhaustion and timeouts. Increasing the pool size to 20 with appropriate max_overflow settings resolves the errors and meets the service-level objectives in the sandbox environment. This fix can be implemented by adjusting the PAYMENT_DB_POOL_SIZE to 20 and max_overflow to the previous release's value, with rollback to the previous release or pool size 5 if issues arise. *(confidence 0.98; evidence cited: EVD-421, EVD-E969AB54, EVD-20E68B2D, EVD-B28F2E14, EVD-422)*

Test engineering · iteration 1 · AI-GENERATED / Foundry agent behave-testing v4

Tool	Input	Result
`get_service_status`	-	status read
`list_changes`	-	5 changes

Finding: The root cause of the GET /payments/summary endpoint failures under load in Payment API v3.8.1 is the reduction of the DB connection pool size to 5 with no overflow, causing 18% HTTP 500 errors and high p95 latency under 50 concurrent requests. Increasing the pool size to 20 restores zero 5xx errors and meets the p95 latency SLO. Regression tests should simulate 50 concurrent requests targeting zero 5xx and p95 latency under 2 seconds, matching the successful pool size 20 configuration measurements. Approved remediation options include restoring the pool size to 20 or rollback to the previous release, which lacks Feature X. *(confidence 0.95; evidence cited: EVD-421, EVD-422, EVD-E969AB54, EVD-20E68B2D, EVD-B28F2E14)*

Change risk · iteration 1 · AI-GENERATED / Foundry agent behave-risk v4

Tool	Input	Result
`list_changes`	-	7 changes
`get_service_status`	-	status read

Finding: The Payment API v3.8.1 release caused HTTP 500 errors in the /payments/summary endpoint due to a reduced database connection pool size of 5 with no overflow, leading to timeouts under 50 concurrent requests (18% failure rate). Increasing the pool size back to 20 eliminates errors and meets the service-level objectives. The blast radius is limited to the Payment API service in the sandbox environment with degraded health observed before the pool size increase. Change risk for reverting to pool size 20 is medium with a clear rollback path and active monitoring recommended. *(confidence 0.95; evidence cited: EVD-421, EVD-E969AB54, EVD-20E68B2D, EVD-B28F2E14)*

Release coordination · iteration 1 · AI-GENERATED / Foundry agent behave-release v4

Tool	Input	Result
`list_changes`	-	7 changes
`get_service_status`	-	status read

Finding: The Payment API sandbox is currently running release v3.8.1 with an active database connection pool size of 20, which eliminates the HTTP 500 errors previously seen with a pool size of 5 under load. Multiple completed changes and successful deployments confirm the fix has been applied. The service health remains degraded pending full verification. *(confidence 0.95; evidence cited: EVD-E969AB54, EVD-B28F2E14)*

4. Issues found and what was solved

Item	What we found	Status
Root cause	The root cause of the Payment API v3.8.1 /payments/summary endpoint failures under concurrent load is the reduction of the database connection pool size from 20 to 5, with no overflow and a 2-second timeout, causing frequent QueuePool timeouts and HTTP 500 errors (18% failure rate) under 50 concurrent requests. Increasing the pool size back to 20 fully eliminates the errors and meets the sandbox service-level target. This is supported by evidence from configuration (EVD-421), workload metrics showing failures at pool size 5 and no failures at pool size 20 (EVD-E969AB54, EVD-20E68B2D, EVD-B28F2E14), and documented recovery options (EVD-422, KB-108).	Solved and confirmed
Fix applied to the sandbox	Requested release 3.8.1 with pool size 20; running service reports {'service': 'payment-api', 'release': '3.8.1', 'pool_size': 20, 'max_overflow': 0, 'pool_timeout': 2, 'hold_seconds': 0.3}.	Applied and verified
Measurement EVD-E969AB54	Measured workload: 9/50 requests failed (pool 5). 9/50 requests failed with HTTP 5xx (18.0% error rate) at 50 concurrent requests; p95 2444 ms, max 2746 ms; release 3.8.1, pool size 5; Feature X (GET /payments/summary) available.	Recorded
Measurement EVD-20E68B2D	Measured workload: 0/50 requests failed (pool 20). 0/50 requests failed with HTTP 5xx (0.0% error rate) at 50 concurrent requests; p95 962 ms, max 963 ms; release 3.8.1, pool size 20; Feature X (GET /payments/summary) available.	Recorded

Item	What we found	Status
Measurement EVD-B28F2E14	Measured workload: 0/50 requests failed (pool 20). 0/50 requests failed with HTTP 5xx (0.0% error rate) at 50 concurrent requests; p95 956 ms, max 957 ms; release 3.8.1, pool size 20; Feature X (GET /payments/summary) available.	Recorded
Measurement EVD-8D835B12	Measured workload: 0/50 requests failed (pool 20). 0/50 requests failed with HTTP 5xx (0.0% error rate) at 50 concurrent requests; p95 943 ms, max 946 ms; release 3.8.1, pool size 20; Feature X (GET /payments/summary) available.	Recorded

5. Human decisions (every key step needed a person)

When	Gate	Decision	By	Note	Time (UTC)
Iteration 1	team	approve	Alex Morgan	-	2026-11-13 10:59:07
Iteration 1	root-cause	approve	Alex Morgan	-	2026-11-13 10:59:26
Iteration 1	fix	approve	Sam Rivera	-	2026-11-13 10:59:39
Iteration 1	resolution	resolve	Alex Morgan	Reviewed the measured outcome	2026-11-13 10:59:52

6. Final validation

MEASURED: 0/50 requests failed with HTTP 5xx (0.0% error rate) at 50 concurrent requests; p95 943 ms, max 946 ms; release 3.8.1, pool size 20; Feature X (GET /payments/summary) available.

7. Learnings carried between iterations and open items

• Resolved without another iteration, so there were no learnings to carry forward.

All findings are AI-generated proposals for human review. Agents can read but never change anything; every change in this mission was approved by a person.

8. Mission timeline (latest events)

Time	Actor	Event
10:58:55	Alex Morgan	Mission started by Alex Morgan for INC-1043: Payment summary endpoint failing after v3.8.1 release
10:58:55	Orchestrator	Iteration 1: the Orchestrator is analysing the ticket.
10:59:06	Orchestrator	Proposed a team of 9: Intake & classification, Log analysis, Observability correlation, Repository analysis, Root-cause analysis, Remediation, Test engineering, Change risk, Release coordination.
10:59:07	Alex Morgan	Alex Morgan approved a team of 9.
10:59:09	Orchestrator	Investigate: Intake & classification, Log analysis, Observability correlation, Repository analysis started work.
10:59:18	Intake & classification	Intake & classification investigated with 2 read-only tool calls: get_ticket() → ticket read; list_evidence() → 5 evidence
10:59:18	Intake & classification	Intake & classification: The Payment API v3.8.1 release introduced Feature X (GET /payments/summary) with a DB connection pool size reduced to 5, causing HTTP 500 errors under 50 concurrent requests due to pool timeouts (9/50 requests failed). Increasing the pool size to 20 eliminates the failures and meets the sandbox SLO of zero HTTP 5xx and p95 under 2 seconds. Severity High classification is appropriate given the impact and failure rate. Recommended recovery includes increasing pool size to 20 or rolling back the release.
10:59:18	Log analysis	Log analysis investigated with 3 read-only tool calls: list_evidence() → 5 evidence; search_logs(TimeoutError) → 0 matches; search_logs(QueuePool) → 0 matches

Time	Actor	Event
10:59:18	Log analysis	Log analysis: The Payment API v3.8.1 release introduced Feature X and reduced the DB connection pool size to 5 with no overflow and a 2s timeout. Under 50 concurrent requests, this configuration caused an 18% HTTP 500 failure rate correlated with connection pool timeouts, while increasing the pool size to 20 resolved the errors. The root cause is the insufficient DB connection pool size leading to queue pool timeout errors during concurrent load.
10:59:18	Observability correlation	Observability correlation investigated with 3 read-only tool calls: get_service_status() → status read; search_logs(QueuePool) → 0 matches; search_logs(TimeoutError) → 0 matches
10:59:18	Observability correlation	Observability correlation: The high error rate (18%) and elevated p95 latency (~2444 ms) for GET /payments/summary under 50 concurrent requests are caused by connection pool saturation. The Payment API v3.8.1 was configured with PAYMENT_DB_POOL_SIZE=5 and max_overflow=0, leading to frequent connection timeouts (QueuePool limit reached). Increasing the pool size to 20 eliminates the 5xx errors and reduces latency to ~960 ms, meeting service targets. The root cause is the insufficient DB connection pool size for the new feature workload on v3.8.1.
10:59:18	Repository analysis	Repository analysis investigated with 4 read-only tool calls: list_changes() → 5 changes; list_repo_files() → 4 files; read_repo_file(sandbox.env.example) → sandbox.env.example read; read_repo_file(app.py) → app.py read
10:59:18	Repository analysis	Repository analysis: The Payment API v3.8.1 release introduced Feature X which requires holding a DB connection for approximately 0.3 seconds per request. The database connection pool size was reduced to 5 with max_overflow set to 0 and pool_timeout to 2 seconds. Under a 50 concurrent request workload, this configuration leads to an 18% HTTP 500 error rate caused by SQLAlchemy QueuePool timeouts, whereas increasing the pool size to 20 eliminates these failures. This suggests that the root cause of the endpoint failures under load is the insufficient connection pool size configuration in v3.8.1, correlating directly with the onset of errors after deployment.
10:59:18	Orchestrator	Handing Investigate findings to Diagnose.
10:59:19	Orchestrator	Diagnose: Root-cause analysis started work.
10:59:26	Root-cause analysis	Root-cause analysis investigated with 6 read-only tool calls: list_evidence() → 5 evidence; search_logs(TimeoutError) → 0 matches; list_changes() → 5 changes; get_service_status() → status read; list_repo_files() → 4 files; search_knowledge(connection pool timeout) → 2 articles
10:59:26	Root-cause analysis	Root-cause analysis: The root cause of the Payment API v3.8.1 /payments/summary endpoint failures under concurrent load is the reduction of the database connection pool size from 20 to 5, with no overflow and a 2-second timeout, causing frequent QueuePool timeouts and HTTP 500 errors (18% failure rate) under 50 concurrent requests. Increasing the pool size back to 20 fully eliminates the errors and meets the sandbox service-level target. This is supported by evidence from configuration (EVD-421), workload metrics showing failures at pool size 5 and no failures at pool size 20 (EVD-E969AB54, EVD-20E68B2D, EVD-B28F2E14), and documented recovery options (EVD-422, KB-108).
10:59:26	Orchestrator	Handing Diagnose findings to Remediate & validate.
10:59:26	Alex Morgan	Alex Morgan approved: Approve the root cause.
10:59:28	Orchestrator	Remediate & validate: Remediation, Test engineering started work.
10:59:38	Remediation	Remediation investigated with 6 read-only tool calls: list_changes() → 5 changes; get_service_status() → status read; read_repo_file(config.py) → File 'config.py' is not available. Use list_repo_files.; list_repo_files() → 4 files; read_repo_file(sandbox.env.example) → sandbox.env.example read; read_repo_file(app.py) → app.py read
10:59:38	Remediation	Remediation: The root cause of the Payment API v3.8.1 /payments/summary endpoint failures (HTTP 500) under concurrent load is due to the database connection pool size being set to 5 with max_overflow 0, leading to connection pool exhaustion and timeouts. Increasing the pool size to 20 with appropriate max_overflow settings resolves the errors and meets the service-level objectives in the sandbox environment. This fix can be implemented by adjusting the PAYMENT_DB_POOL_SIZE to 20 and max_overflow to the previous release's value, with rollback to the previous release or pool size 5 if issues arise.
10:59:38	Test engineering	Test engineering investigated with 2 read-only tool calls: get_service_status() → status read; list_changes() → 5 changes

Time	Actor	Event
10:59:38	Test engineering	Test engineering: The root cause of the GET /payments/summary endpoint failures under load in Payment API v3.8.1 is the reduction of the DB connection pool size to 5 with no overflow, causing 18% HTTP 500 errors and high p95 latency under 50 concurrent requests. Increasing the pool size to 20 restores zero 5xx errors and meets the p95 latency SLO. Regression tests should simulate 50 concurrent requests targeting zero 5xx and p95 latency under 2 seconds, matching the successful pool size 20 configuration measurements. Approved remediation options include restoring the pool size to 20 or rollback to the previous release, which lacks Feature X.
10:59:38	Orchestrator	Handing Remediate & validate findings to Govern & release.
10:59:39	Sam Rivera	Sam Rivera approved: Approve the fix for release.
10:59:41	Release coordination	Applied the human-approved fix to the live Payment API sandbox: release 3.8.1, pool size 20 (verified on the running service).
10:59:42	Orchestrator	Govern & release: Change risk, Release coordination started work.
10:59:48	Change risk	Change risk investigated with 2 read-only tool calls: list_changes() → 7 changes; get_service_status() → status read
10:59:48	Change risk	Change risk: The Payment API v3.8.1 release caused HTTP 500 errors in the /payments/summary endpoint due to a reduced database connection pool size of 5 with no overflow, leading to timeouts under 50 concurrent requests (18% failure rate). Increasing the pool size back to 20 eliminates errors and meets the service-level objectives. The blast radius is limited to the Payment API service in the sandbox environment with degraded health observed before the pool size increase. Change risk for reverting to pool size 20 is medium with a clear rollback path and active monitoring recommended.
10:59:48	Release coordination	Release coordination investigated with 2 read-only tool calls: list_changes() → 7 changes; get_service_status() → status read
10:59:48	Release coordination	Release coordination: The Payment API sandbox is currently running release v3.8.1 with an active database connection pool size of 20, which eliminates the HTTP 500 errors previously seen with a pool size of 5 under load. Multiple completed changes and successful deployments confirm the fix has been applied. The service health remains degraded pending full verification.
10:59:50	Validation	Outcome: resolved. MEASURED: 0/50 requests failed with HTTP 5xx (0.0% error rate) at 50 concurrent requests; p95 943 ms, max 946 ms; release 3.8.1, pool size 20; Feature X (GET /payments/summary) available.
10:59:52	Alex Morgan	Alex Morgan confirmed the issue is resolved. Reviewed the measured outcome